

UNIVERSITY OF INSUBRIA

Department of Theoretical and Applied Sciences – DiSTA
Master of Science in Computer Science

LOGIC FOR ARTIFICIAL INTELLIGENCE

Notes

Author:
Roberto Vicario

Student ID:
744072

Academic Year 2025-2026

Contents

1 Introduction **3**
1.1 Agents . . . 3

2 Searching Problems **5**
2.1 Problem Formulation . . . 5
2.1.1 Evaluation . . . 6
2.2 Uninformed Search . . . 6
2.2.1 Breadth-First Search (BFS) . . . 7
2.2.2 Uniform-Cost Search (UCS) . . . 8
2.2.3 Depth-First Search (DFS) . . . 9
2.2.4 Depth-Limited Search (DLS) . . . 10
2.2.5 Iterative-Deepening Search (IDS) . . . 11
2.2.6 Bidirectional Search . . . 12
2.3 Informed Search . . . 13
2.3.1 A* Search . . . 14
2.3.2 IDA* (Iterative-Deepening A*) . . . 16
2.3.3 Recursive Best-First Search (RBFS) . . . 17

3 Searching Problems in Complex Environments **19**
3.1 Local Search . . . 19
3.1.1 Hill Climbing . . . 19
3.1.2 Simulated Annealing . . . 20
3.1.3 Local Beam Search . . . 21
3.1.4 Evolutionary Algorithms . . . 22
3.1.5 Continuous Search Spaces . . . 22
3.2 Search with Nondeterministic Actions . . . 23
3.2.1 AND-OR Search . . . 23
3.3 Search with Incomplete Observation . . . 24
3.3.1 Search with No Observation . . . 24
3.3.2 Search with Partial Observation . . . 25
3.4 Online Search . . . 26
3.4.1 Online Depth-First Search . . . 26
3.4.2 Online Hill Climbing . . . 27
3.4.3 Random Walk . . . 27
3.4.4 Learning Real-Time A* (LRTA*) . . . 28

4 Adversarial Search and Game Playing **29**
4.1 Adversarial Search . . . 29
4.1.1 Minimax Search . . . 29
4.1.2 Alpha-Beta Pruning . . . 31
4.1.3 Heuristic Minimax (H-Minimax) . . . 32
4.1.4 Monte Carlo Tree Search (MCTS) . . . 33
4.2 Stochastic Games . . . 34
4.2.1 Expectiminimax . . . 34
4.3 Partially Observable Games . . . 35
4.3.1 Averaging over Clairvoyance . . . 35
4.4 Search Strategies: Summary . . . 36

5 Constraint Satisfaction **37**
5.1 Problem Formulation . . . 37
5.1.1 Constraints . . . 37
5.1.2 Assignments and Solutions . . . 37
5.1.3 Examples . . . 38
5.2 Types of CSP . . . 39
5.2.1 Domains . . . 39
5.2.2 Constraint Arity . . . 39
5.3 Local Search . . . 40
5.3.1 Min-Conflicts . . . 40
5.4 Constraint Propagation . . . 41
5.4.1 Node Consistency . . . 41
5.4.2 Arc Consistency . . . 42
5.4.3 AC-3 . . . 42
5.4.4 Stronger Consistency . . . 43
5.5 Backtracking Search . . . 44
5.5.1 Backtracking Algorithm . . . 44
5.5.2 Variable Ordering . . . 45
5.5.3 Value Ordering . . . 45
5.5.4 Inference . . . 45
5.5.5 Advanced Techniques . . . 46
5.6 Exploiting Problem Structure . . . 46
5.6.1 Connected Components . . . 46
5.6.2 Tree-Structured CSPs . . . 46
5.6.3 Cutset Conditioning . . . 48
5.6.4 Tree Decomposition . . . 48

1 Introduction

Artificial Intelligence (AI) studies how to build systems that perform tasks requiring intelligence. This course focuses on ***SYMBOLIC AI***: explicit knowledge representations and reasoning.

1.1 Agents

An agent perceives an environment through ***SENSORS*** and acts through ***ACTUATORS***. The environment can be physical or virtual.

A ***PERCEPT*** $p \in P$ is an observation. A percept sequence $[p_0, \dots, p_t] \in P^*$ is the observation history.

An ***AGENT FUNCTION*** maps percept histories to actions:

$$\boxed{f : P^* \to A}$$

An ***AGENT PROGRAM*** implements this function on a computing architecture. The mathematical function is not necessarily implementable.

A ***PERFORMANCE MEASURE*** evaluates sequences of environment states, rather than percepts alone.

A ***RATIONAL AGENT*** chooses actions expected to maximize performance, given its observations and built-in knowledge. Rationality does not imply omniscience or guarantee success: exploration and learning may improve future decisions.

Limited resources require ***LIMITED RATIONALITY***. Objectives must be specified carefully: optimizing the wrong measure can produce unwanted behaviour.

> ***PEAS***

Before designing, implementing and evaluating agents we need to specify four things (recognized by the acronym PEAS):

- **Performance Measure:** What is the agent trying to do?
- **Environment:** What is it like?
- **Actuators:** What can the agent do?
- **Sensors:** What can the agent “see”?

> ***Properties of Environments***

Property	Meaning
Fully / partially / unobservable	all / some / no relevant state information is perceived
Single / multi-agent	one agent / multiple cooperating or competing agents
Deterministic / nondeterministic	state and actions fix the next state / allow multiple outcomes
Episodic / sequential	independent decisions / actions affect future decisions
Static / dynamic	unchanged / may change during deliberation
Discrete / continuous	distinct countable values / continuous values
Known / unknown	rules are known / must be learned through experience

> ***Architectures***

Architectures can be classified by the type of agent program they support:

- **Simple reflex:** choose from the current percept using condition-action rules. Efficient, but cannot remember past observations.
- **Model-based:** maintain an internal state, updated through percepts and a model of action effects and world changes.
- **Goal-based:** predict action outcomes and choose actions that reach a goal, using search or planning.
- **Utility-based:** compare outcome desirability and maximize expected utility, handling trade-offs and uncertainty.

2 Searching Problems

A searching problem can be formulated as a search for a path from an initial state to a goal state. This formulation assumes the environment is fully observable, deterministic, static, and discrete.

2.1 Problem Formulation

A search problem is defined as:

$$\boxed{\mathcal{P} = (S, s_0, G, A, T, c)}$$

where:

- S is the set of states.
- $s_0 \in S$ is the initial state.
- $G \subseteq S$ is the set of goal states.
- $A(s)$ is the set of applicable actions in state s.
- $T(s, a) \in S$, for $a \in A(s)$, is the successor state.
- $c(s, a, s')$ is the cost of transitioning from s to s' via a.

Assuming A is the global action set and T is restricted to applicable state-action pairs, ***DETERMINISM*** follows from the fact that:

$$T : S \times A \to S$$

is a single-valued function.

> ***Paths***

A path is a sequence:

$$\pi = (s_0, a_0, s_1, a_1, \dots, a_{n-1}, s_n)$$

such that:

$$\forall i \in \{0, \dots, n-1\} : \quad a_i \in A(s_i), \quad s_{i+1} = T(s_i, a_i)$$

A path is a solution if:

$$s_n \in G$$

Its total cost is:

$$\boxed{C(\pi) = \sum_{i=0}^{n-1} c(s_i, a_i, s_{i+1})}$$

A solution π^* is ***OPTIMAL*** if:

$$\boxed{\pi^* \in \arg\min_{\pi \in \Pi_G} C(\pi)}$$

where Π_G denotes the set of all solution paths.

2.1.1 Evaluation

A search algorithm $\mathcal{A}$ is evaluated according to four criteria:

- **Completeness:** whether $\mathcal{A}$ finds a solution whenever one exists.
- **Optimality:** whether $\mathcal{A}$ returns a minimum-cost solution.
- **Time Complexity:** number of nodes generated or expanded.
- **Space Complexity:** maximum number of nodes stored in memory.

We will use b for the branching factor, d for the depth of the shallowest solution, and m for the maximum depth of the search tree.

2.2 Uninformed Search

A search is uninformed if it uses only the problem definition, without any estimate of how close a state is to the goal.

The Python pseudocode uses the same abstract operations as the slides: `Node` creates a search node, `expand` generates its children, and `is_cycle` checks its ancestor path. Nodes store `state`, path cost `g` (initially zero), and `depth` (initially zero). `PriorityQueue` pops the node with the smallest key; `peek` reads it without removal. `None` means failure; `"cutoff"` means the depth limit was reached. Heuristics `h` take a node. For bidirectional search, `join_nodes` joins the two paths in forward order, and `terminated` checks the stopping condition for the chosen search strategy.

2.2.1 Breadth-First Search (BFS)

The algorithm expands nodes in order of increasing depth, exploring the shallowest nodes first. It uses a FIFO queue to store the frontier.

If the shallowest solution is at depth d, the number of nodes grows as:

$$1 + b + b^2 + \cdots + b^d = O(b^d)$$

> ***Pseudocode***

```python
def bfs(problem):
    node = Node(problem.initial)
    if problem.is_goal(node.state):
        return node
    frontier, reached = deque([node]), {node.state}
    while frontier:
        for child in expand(problem, frontier.popleft()):
            if problem.is_goal(child.state):
                return child
            if child.state not in reached:
                reached.add(child.state)
                frontier.append(child)
    return None
```

> ***Evaluation***

Criterion	Value
Complete	Yes[1]
Optimal	Yes[3]
Space	$O(b^d)$
Time	$O(b^d)$
Advantages	finds shallowest solution
Disadvantages	high memory usage

Assumptions:

- (1) complete if b is finite, and the state space either has a solution or is finite.
- (3) cost-optimal if action costs are identical.

2.2.2 Uniform-Cost Search (UCS)

The algorithm expands nodes in order of increasing path cost, exploring the cheapest paths first. It uses a priority queue ordered by $g(n)$ to store the frontier.

It is equivalent to Dijkstra's algorithm when applied to shortest-path search with non-negative edge costs:

$$f(n) = g(n)$$

where $g(n)$ is the cost from the initial state to node n.

> ***Pseudocode***

```python
def best_first_search(problem, f):
    node = Node(problem.initial)
    frontier = PriorityQueue([node], key=f)
    reached = {node.state: node}
    while frontier:
        node = frontier.pop()
        if problem.is_goal(node.state):
            return node
        for child in expand(problem, node):
            s = child.state
            if s not in reached or child.g < reached[s].g:
                reached[s] = child
                frontier.add(child)
    return None

def ucs(problem):
    return best_first_search(problem, lambda n: n.g)
```

> ***Evaluation***

Criterion	Value
Complete	Yes[1,2]
Optimal	Yes
Space	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$
Time	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$

Criterion	Value
Advantages	finds the least-cost solution
Disadvantages	high time and memory usage

Assumptions:

- (1) complete if b is finite, and the state space either has a solution or is finite.
- (2) complete if all action costs are $\geq \varepsilon > 0$.

2.2.3 Depth-First Search (DFS)

The algorithm expands nodes in order of decreasing depth, following a branch as far as possible before backtracking. It uses a LIFO stack to store the frontier.

If b is the branching factor and m is the maximum depth, the number of nodes explored can grow exponentially:

$$1 + b + b^2 + \cdots + b^m = O(b^m)$$

> ***Pseudocode***

```
def dfs(problem):
    return dls(problem, float("inf"))
```

> ***Evaluation***

Criterion	Value
Complete	No
Optimal	No
Time	$O(b^m)$
Space	$O(bm)$
Advantages	low memory usage

Criterion	Value
Disadvantages	can get stuck in infinite branches

2.2.4 Depth-Limited Search (DLS)

The algorithm expands nodes in order of decreasing depth, following a branch up to a fixed depth limit ℓ before backtracking. It uses a LIFO stack to store the frontier.

The number of nodes explored is bounded by:

$$1 + b + b^2 + \cdots + b^\ell = O(b^\ell)$$

> ***Pseudocode***

```
def dls(problem, limit):
    frontier = [Node(problem.initial)]
    result = None
    while frontier:
        node = frontier.pop()
        if problem.is_goal(node.state):
            return node
        if node.depth >= limit:
            result = "cutoff"
        elif not is_cycle(node):
            frontier.extend(expand(problem, node))
    return result
```

> ***Evaluation***

Criterion	Value
Complete	No
Optimal	No
Time	$O(b^\ell)$
Space	$O(b\ell)$
Advantages	avoids infinite-depth exploration

Criterion	Value
Disadvantages	may miss solutions beyond the limit

2.2.5 Iterative-Deepening Search (IDS)

The algorithm repeatedly explores the search tree with increasing depth limits, restarting from the initial node until a solution is found. It uses depth-limited DFS at each iteration, storing only the current path and its unexplored alternatives.

The depth limits are:

$$\ell = 0, 1, 2, \dots, d$$

Although nodes are explored multiple times, the asymptotic time complexity remains:

$$1 + b + b^2 + \cdots + b^d = O(b^d)$$

> ***Pseudocode***

```python
def ids(problem):
    for depth in count():
        result = dls(problem, depth)
        if result != "cutoff":
            return result
```

> ***Evaluation***

Criterion	Value
Complete	Yes[1]
Optimal	Yes[3]
Time	$O(b^d)$
Space	$O(bd)$
Advantages	low memory usage

Criterion	Value
Disadvantages	repeated exploration of shallow nodes

Assumptions:

- (1) complete if b is finite, and the state space either has a solution or is finite.
- (3) cost-optimal if action costs are identical.

2.2.6 Bidirectional Search

The algorithm searches forward from the initial state and backward from the goal, looking for a state reached by both searches. It uses a separate frontier for each direction, with a FIFO queue for each when both searches use BFS.

Each search explores approximately half the solution depth:

$$b^{d/2} + b^{d/2} \ll b^d$$

It is applicable when the goal state is known, backward transitions can be generated, and the intersection between the two frontiers can be detected efficiently.

> ***Pseudocode***

```
def bidirectional_search(p_f, f_f, p_b, f_b):
    n_f, n_b = Node(p_f.initial), Node(p_b.initial)
    q_f = PriorityQueue([n_f], key=f_f)
    q_b = PriorityQueue([n_b], key=f_b)
    r_f, r_b = {n_f.state: n_f}, {n_b.state: n_b}
    solution = None
    if n_f.state == n_b.state:
        return n_f
    while not terminated(solution, q_f, q_b):
        if f_f(q_f.peek()) < f_b(q_b.peek()):
            solution = proceed("F", p_f, q_f, r_f, r_b, solution)
        else:
            solution = proceed("B", p_b, q_b, r_b, r_f, solution)
    return solution

def proceed(direction, problem, frontier, reached, other, solution):
    for child in expand(problem, frontier.pop()):
        s = child.state
```

```
        if s not in reached or child.g < reached[s].g:
            reached[s] = child
            frontier.add(child)
            if s in other:
                candidate = join_nodes(direction, child, other[s])
                if solution is None or candidate.g < solution.g:
                    solution = candidate
    return solution
```

> ***Evaluation***

Criterion	Value
Complete	Yes[1,4]
Optimal	Yes[3,4]
Space	$O(b^{d/2})$
Time	$O(b^{d/2})$
Advantages	much faster than BFS for large d
Disadvantages	high memory usage

Assumptions:

- (1) complete if b is finite, and the state space either has a solution or is finite.
- (3) cost-optimal if action costs are identical.
- (4) if both directions are breadth-first or uniform-cost.

2.3 Informed Search

A search is informed if it uses a heuristic function:

$$h : S \to \mathbb{R}_{\geq 0}$$

to guide the search toward promising states, reducing unnecessary exploration.

> ***Admissible Heuristics***

Let $h^*(s)$ be the true minimum cost from s to a goal. A heuristic is ***ADMISSIBLE*** if it never overestimates the actual cost:

$$\boxed{\forall s \in S: \quad 0 \leq h(s) \leq h^*(s)}$$

An admissible heuristic is always optimistic, but not necessarily accurate.

2.3.1 A* Search

The algorithm expands nodes in order of increasing estimated total cost, exploring the most promising paths first. It uses a priority queue ordered by $f(n)$ to store the frontier.

The estimated total cost is:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the cost from the initial state to node n.
- $h(n)$ is the heuristic estimate from n to a goal.
- $f(n)$ is the estimated total cost of a solution through n.

> ***Pseudocode***

```
def a_star(problem, h):
    return best_first_search(problem, lambda n: n.g + h(n))
```

> ***Evaluation***

Criterion	Value
Complete	Yes
Optimal	Yes[1]
Advantages	heuristic-guided exploration
Disadvantages	high memory usage

Assumptions:

- (1) optimal if h is admissible.

> ***PROOF (1)***

Assume h is admissible and A* returns a suboptimal solution of cost $C > C^*$, where C^* is the optimal cost.

Before selecting that suboptimal goal, there must be a node n on an optimal path still in the frontier, reached with its optimal cost $g^*(n)$.

Since A* selects the node with minimum f, selecting the suboptimal goal implies:

$$C \leq f(n)$$

However, by definition:

$$f(n) = g^*(n) + h(n)$$

Since h is admissible:

$$h(n) \leq h^*(n)$$

Therefore, because n lies on an optimal path:

$$f(n) \leq g^*(n) + h^*(n) = C^*$$

Combining the inequalities:

$$\boxed{C \leq f(n) \leq C^* < C}$$

This is ***IMPOSSIBLE***.

Hence A* cannot return a suboptimal solution under the stated assumptions.

$$\boxed{C = C^*}$$

2.3.2 IDA* (Iterative-Deepening A*)

The algorithm repeatedly explores the search tree with increasing cost limits, skipping nodes whose estimated total cost exceeds the current limit. It uses depth-first search at each iteration, storing only the current path and its unexplored alternatives.

The estimated total cost is:

$$f(n) = g(n) + h(n)$$

If no solution is found, the cost limit increases to the smallest f-value that exceeded the previous limit.

> ***Pseudocode***

```python
def ida_star(problem, h):
    def search(node, limit):
        f = node.g + h(node)
        if f > limit:
            return None, f
        if problem.is_goal(node.state):
            return node, f
        next_limit = float("inf")
        if not is_cycle(node):
            for child in expand(problem, node):
                result, cost = search(child, limit)
                if result is not None:
                    return result, cost
                next_limit = min(next_limit, cost)
        return None, next_limit

    root = Node(problem.initial)
    limit = h(root)
    while limit < float("inf"):
        result, limit = search(root, limit)
        if result is not None:
            return result
    return None
```

> ***Evaluation***

Criterion	Value
Complete	Yes
Optimal	Yes[1]
Advantages	low memory usage
Disadvantages	high time usage

Assumptions:

- (1) optimal if h is admissible.

2.3.3 Recursive Best-First Search (RBFS)

The algorithm follows the path with the lowest estimated total cost, backtracking when another path becomes more promising. It uses recursive depth-first search, remembering the best alternative cost at each level to decide when to backtrack.

> ***Pseudocode***

```
def rbfs(problem, h):
    def search(node, limit):
        if problem.is_goal(node.state):
            return node, node.f
        successors = list(expand(problem, node))
        if not successors:
            return None, float("inf")
        for child in successors:
            child.f = max(child.g + h(child), node.f)
        while True:
            successors.sort(key=lambda n: n.f)
            best = successors[0]
            if best.f > limit or best.f == float("inf"):
                return None, best.f
            alternative = (successors[1].f if len(successors) > 1
                           else float("inf"))
            result, best.f = search(best, min(limit, alternative))
            if result is not None:
                return result, best.f

    root = Node(problem.initial)
    root.f = h(root)
    return search(root, float("inf"))[0]
```

> ***Evaluation***

Criterion	Value
Complete	Yes
Optimal	Yes[1]
Advantages	low memory usage
Disadvantages	high time usage

Assumptions:

- (1) optimal if h is admissible.

3 Searching Problems in Complex Environments

3.1 Local Search

A local search problem asks for a configuration that maximizes an objective function. The solution is the configuration itself, regardless of the path used to reach it.

The problem is defined by:

- S: the set of configurations.
- $N(s)$: the neighbours of configuration s.
- $V : S \to \mathbb{R}$: the objective function to maximize.

For example, the n-queens problem asks for a placement of n queens on an $n \times n$ chessboard such that no queen attacks another. If $c(s)$ counts conflicts, a possible objective is:

$$\boxed{V(s) = -c(s)}$$

Local search usually stores only the current configuration, or a small set of configurations.

3.1.1 Hill Climbing

The algorithm selects the neighbour with the highest objective value, stopping when no neighbour improves the current state. It stores only the current state and evaluates its neighbours at each step.

The update rule is:

$$s' \in \arg\max_{u \in N(s)} V(u), \qquad s \leftarrow s' \quad \text{if } V(s') > V(s).$$

> ***Pseudocode***

Here `neighbours` generates nearby configurations, and `value` is the objective function.

```
def hill_climbing(state, neighbours, value):
    while True:
        next_state = max(neighbours(state), key=value, default=state)
        if value(next_state) <= value(state):
            return state
        state = next_state
```

Evaluation

Criterion	Value
Complete	No, in general
Optimal	No, in general
Advantages	simple; low memory usage
Disadvantages	can stop before reaching a global maximum

Local Optima

Hill climbing can get stuck in:

- **Local maximum:** a state better than its neighbours, but worse than the global maximum.
- **Plateau:** a region where neighbouring states have the same objective value.
- **Ridge:** a region where progress requires a sequence of moves that are not individually improving.

Variants

- **Sideways moves:** allow moves with unchanged objective value to cross plateaus.
- **Stochastic:** randomly select an improving neighbour.
- **First-choice:** generate random neighbours until an improving one is found.
- **Random-restart:** restart from a random state when progress stops, keeping the best state found.

3.1.2 Simulated Annealing

The algorithm selects a random neighbour, accepting improvements and sometimes accepting worse states. It stores only the current state and uses a decreasing temperature to control the probability of accepting worse moves.

Let:

$$\Delta = V(s') - V(s).$$

The acceptance probability is:

$$\boxed{P(\text{accept } s') = \begin{cases} 1, & \Delta \geq 0, \\ e^{\Delta/T}, & \Delta < 0. \end{cases}}$$

where $T > 0$ is the ***TEMPERATURE***.

- **High temperature:** worse moves are more likely, allowing exploration.
- **Low temperature:** worse moves are less likely, favouring improvement.

> ***Evaluation***

Criterion	Value
Advantages	can escape local maxima and plateaus
Disadvantages	performance depends on the cooling schedule

3.1.3 Local Beam Search

The algorithm generates the neighbours of all current states, keeping the k best configurations for the next step. It stores a set of $k > 1$ states and selects their successors together.

If the current set is B_t, the next set is:

$$B_{t+1} = \mathrm{Top}_k \left(\bigcup_{s \in B_t} N(s) \right).$$

where Top_k selects the k configurations with the highest objective values.

> ***Evaluation***

Criterion	Value
Advantages	shares information between search paths
Disadvantages	states can become too similar, reducing diversity

Stochastic variants select successors probabilistically to preserve diversity.

3.1.4 Evolutionary Algorithms

The algorithm generates new configurations by combining and randomly modifying selected configurations. It stores a population of candidates, using fitness to guide the selection of parents.

The main steps are:

1. Generate a random initial population.
2. Select parents according to their fitness.
3. Combine parents through ***CROSSOVER*** to generate children.
4. Randomly modify children through ***MUTATION***.
5. Optionally retain the best parents through ***ELITISM***.

Repeat until the stopping condition is satisfied, then return the best configuration found. The slides use lack of further fitness improvement as a stopping condition.

> ***Evaluation***

Criterion	Value
Advantages	can combine useful parts of different configurations
Disadvantages	depends strongly on representation and crossover

A useful crossover should preserve good contributions from both parents. For numerical configurations, averaging may be more meaningful than mixing their binary encodings.

3.1.5 Continuous Search Spaces

A continuous configuration is represented as:

$$x = (x_1, \dots, x_n) \in \mathbb{R}^n.$$

If the objective function is sufficiently smooth, optimization can use:

- **Gradient ascent/descent:** move in the direction of increasing/decreasing objective value.
- **Newton–Raphson:** use local derivative information to compute updates.

With constraints, the problem becomes ***CONSTRAINED OPTIMIZATION***. Convex problems admit efficient methods under suitable assumptions.

If the objective and constraints are linear, the problem is ***LINEAR PROGRAMMING***. Polynomial-time algorithms exist; the simplex method is widely used, but has exponential worst-case complexity.

3.2 Search with Nondeterministic Actions

An action is nondeterministic if it can produce more than one successor state. Its possible outcomes are represented by:

$$\boxed{\text{RESULTS}(s,a) \subseteq S, \qquad \text{RESULTS}(s,a) \neq \emptyset}$$

A solution must specify what to do after each possible outcome. This is a ***CONDITIONAL PLAN***.

For example, in the nondeterministic Vacuum World:

$$\text{RESULTS}(1, \mathsf{suck}) = \{5, 8\}.$$

A conditional plan from state 1 is:

```
[suck, if state = 5 then [right, suck] else []]
```

3.2.1 AND-OR Search

The algorithm searches for an action whose possible outcomes can all be solved, building a conditional plan. It uses OR-nodes for action choices and AND-nodes for their possible outcomes.

The two node types are:

- **OR-node:** choose at least one solvable successor.
- **AND-node:** solve every successor.

> ***Solution***

A solution is a subtree such that:

1. It contains the root.
2. Every leaf is a goal.
3. Every OR-node includes at least one successor.

4. Every AND-node includes all successors.

If an action has outcomes $s_1, \dots, s_k$, with plans $p_1, \dots, p_k$, combine them as:

```
if state = s1 then p1
else if state = s2 then p2
...
else if state = sk then pk
```

Depth-first, breadth-first and heuristic search ideas can be adapted to AND-OR trees. In the depth-first method from the slides, a state repeated on the current path causes that branch to fail.

Assumptions:

- This method searches for finite, acyclic conditional plans. Plans that rely on repeatedly trying an action require a different treatment of cycles.

3.3 Search with Incomplete Observation

When the agent cannot identify its physical state, it searches over sets of possible states. Each set is a ***BELIEF STATE***:

$$\boxed{b \subseteq S}$$

3.3.1 Search with No Observation

A sensorless agent must find an action sequence that reaches a goal regardless of the actual initial state.

The search problem is defined by:

- **States:** belief states $b \subseteq S$.
- **Initial state:** all physical states compatible with the initial information.
- **Actions:** actions available in at least one possible state.
- **Transitions:** all possible successors of the current belief state.
- **Goal:** every remaining possible state is a goal.

Assuming illegal actions leave the state unchanged:

$$A(b) = \bigcup_{s \in b} A(s).$$

Extend $T(s, a) = s$ to actions illegal in s. For deterministic physical transitions:

$$\boxed{T(b, a) = \{T(s, a) \mid s \in b\}}$$

The goal condition is:

$$\boxed{b \subseteq G}$$

For nondeterministic physical transitions, take the union of all possible outcomes instead.

> ***Evaluation***

- With N physical states, there are up to 2^N belief states.
- Not every belief state is reachable or relevant.
- If a plan solves b, the same plan solves every $b' \subseteq b$.
- Logical representations can be more compact than explicit sets.
- Action costs may use worst-case or expected cost, depending on the problem.

An alternative is ***INCREMENTAL BELIEF-STATE SEARCH***: find a plan for one possible initial state, check it against the others, and refine the search when it fails.

3.3.2 Search with Partial Observation

With partial observation, an action changes the physical state and a percept reduces uncertainty. The belief state is updated through three steps:

1. **Predict:** compute possible states after the action.
2. **Percept:** determine which observations are possible.
3. **Update:** retain only states compatible with the received observation.

Formally:

$$\begin{aligned} \text{Predict}(b, a) &= \hat{b} = \{T(s, a) \mid s \in b\}, \\ \text{PossiblePercepts}(\hat{b}) &= \{\text{Percept}(s) \mid s \in \hat{b}\}, \\ \text{Update}(\hat{b}, o) &= \{s \in \hat{b} \mid \text{Percept}(s) = o\}. \end{aligned}$$

Different percepts produce different successor belief states:

$$\boxed{\text{RESULTS}(b, a) = \{\text{Update}(\hat{b}, o) \mid o \in \text{PossiblePercepts}(\hat{b})\}}$$

where $\hat{b} = \text{Predict}(b, a)$.

Actions are therefore nondeterministic at the belief-state level, even when physical transitions are deterministic. AND-OR search can build a plan for every possible percept.

3.4 Online Search

An online agent alternates computation, observation and action while exploring an unknown environment. It learns transition outcomes and costs by executing actions.

Assumptions:

- The environment is deterministic and observable.
- $A(s)$ gives the legal actions in the current state.
- The successor s' and cost $c(s, a, s')$ become known after trying a.
- The agent can test whether its current state is a goal.

> ***Competitive Ratio***

The cost of exploration is measured by:

$$\boxed{\text{Competitive ratio} = \frac{\text{total cost of the executed path}}{\text{cost of an optimal path}}}$$

The executed path includes exploration and backtracking. Even with reversible actions, an unknown environment can make this ratio arbitrarily large.

If actions are irreversible, the agent may enter a ***DEAD END*** from which the goal is unreachable.

3.4.1 Online Depth-First Search

The algorithm tries unexplored actions, returning to earlier states when no unexplored action remains. It stores observed transitions and the actions still to try in each visited state.

> ***Evaluation***

Criterion	Value
Complete	Yes, under the assumptions below
Optimal	No
Advantages	systematically explores an unknown environment
Disadvantages	competitive ratio can be arbitrarily large

Assumptions:

- A finite reachable state space, reversible actions and a reachable goal.

3.4.2 Online Hill Climbing

The algorithm selects an action that improves its heuristic estimate of distance to the goal. It uses the current state and the heuristic to guide each move.

Unlike offline local search, the agent cannot freely restart from a random state: it must physically execute actions to move elsewhere. A local minimum can therefore block progress.

3.4.3 Random Walk

The algorithm chooses a random untried action when one is available, otherwise choosing a random legal action. It uses random choices to continue exploration when no heuristic improvement is available.

> ***Evaluation***

Criterion	Value
Complete	With probability 1, under the assumptions below
Optimal	No
Advantages	simple; can escape local minima
Disadvantages	expected exploration cost can be exponential

Assumptions:

- A finite reachable state space, reversible actions and a reachable goal.
- Every available action has a positive probability of selection.

3.4.4 Learning Real-Time A* (LRTA*)

The algorithm chooses the action with the lowest estimated cost to the goal, updating its estimates as it explores. It stores observed transitions and a table $H(s)$ of estimated remaining costs.

When a state is first visited:

$$H(s) \leftarrow h(s).$$

After executing a in s and observing s':

1. Record the transition $\mathrm{result}[s, a] = s'$.
2. Update the estimate of the previous state s.
3. Choose the next action from the current state s'.

The update is:

$$\boxed{H(s) \leftarrow \min_{a \in A(s)} \mathrm{LRTA\text{-}Cost}(s, a, \mathrm{result}[s, a])}$$

where:

$$\mathrm{LRTA\text{-}Cost}(s, a, u) = \begin{cases} h(s), & \text{if the successor } u \text{ is unknown,} \\ c(s, a, u) + H(u), & \text{otherwise.} \end{cases}$$

The next action minimizes the same cost estimate at s'. Updating H can make an apparent local minimum less attractive, allowing the agent to leave it.

4 Adversarial Search and Game Playing

4.1 Adversarial Search

An adversarial search problem involves agents with conflicting objectives. The basic model is a two-player, zero-sum game with deterministic actions and perfect information.

Assumptions:

- **MAX** wants to maximize the final score.
- **MIN** wants to minimize the same score.
- The game ends after finitely many moves.

A game is defined by:

- s_0: the initial state.
- $\text{TO-MOVE}(s)$: the player whose turn it is.
- $A(s)$: the legal moves.
- $T(s, a)$: the state after move a.
- $\text{IS-TERMINAL}(s)$: the terminal-state test.
- $\text{UTILITY}(s)$: the final score from MAX's perspective.

4.1.1 Minimax Search

The algorithm selects the move with the best guaranteed outcome, assuming the opponent also plays optimally. It uses recursive depth-first search, propagating maximum values at MAX-nodes and minimum values at MIN-nodes.

The minimax value is:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s), & s \text{ is terminal}, \\ \max_{a \in A(s)} \text{MINIMAX}(T(s,a)), & \text{TO-MOVE}(s) = \text{MAX}, \\ \min_{a \in A(s)} \text{MINIMAX}(T(s,a)), & \text{TO-MOVE}(s) = \text{MIN}. \end{cases}$$

> ***Pseudocode***

As in the slides, the recursive functions return `(value, move)`. Here the root player is MAX, and `game.utility` returns the score from MAX's perspective.

```python
def minimax(game, state):
    return max_value(game, state)[1]

def max_value(game, state):
    if game.is_terminal(state):
        return game.utility(state), None
    value, move = -float("inf"), None
    for action in game.actions(state):
        score, _ = min_value(game, game.result(state, action))
        if score > value:
            value, move = score, action
    return value, move

def min_value(game, state):
    if game.is_terminal(state):
        return game.utility(state), None
    value, move = float("inf"), None
    for action in game.actions(state):
        score, _ = max_value(game, game.result(state, action))
        if score < value:
            value, move = score, action
    return value, move
```

> ***Evaluation***

Criterion	Value
Complete	Yes, for a finite game tree
Optimal	Yes, against an optimal opponent
Time	$O(b^m)$
Space	$O(bm)$; $O(m)$ with successors generated one at a time
Advantages	finds the best guaranteed outcome
Disadvantages	explores an exponentially large game tree

Here b is the branching factor and m is the maximum game-tree depth. In chess, typical estimates are $b \approx 35$ and a game length of about 80 moves, making exhaustive search impractical.

> ***Multiplayer Games***

With more than two players, use $\mathrm{UTILITY}(s, p)$ for each player p and propagate vectors of scores. Each player chooses according to its own score; alliances may arise.

4.1.2 Alpha-Beta Pruning

The algorithm computes the minimax decision, skipping branches that cannot improve the choice. It uses recursive depth-first search with two bounds, α and β, to decide when to stop exploring a branch.

The bounds summarize alternatives already available along the current path:

- α: the best value already available to MAX.
- β: the best value already available to MIN.

Initially:

$$[\alpha, \beta] = [-\infty, +\infty].$$

If the current node has value estimate v:

- **MAX-node:** update $\alpha \leftarrow \max(\alpha, v)$.
- **MIN-node:** update $\beta \leftarrow \min(\beta, v)$.

The pruning condition is:

$$\boxed{\alpha \geq \beta}$$

For example, if MAX can already obtain 3 from one move and MIN can force at most 2 after another, MAX can discard the second move without examining its remaining outcomes.

> ***Evaluation***

Criterion	Value
Result	same minimax value; tied moves may differ
Time, worst ordering	$O(b^m)$
Time, ideal ordering	$O(b^{m/2})$
Time, random ordering	typically $O(b^{3m/4})$, as estimated in the slides
Advantages	skips irrelevant branches without changing the minimax value
Disadvantages	efficiency depends strongly on move ordering

Criterion	Value

Explore promising moves first to increase pruning. Move ordering can use game-specific rules or patterns learned from game data.

4.1.3 Heuristic Minimax (H-Minimax)

The algorithm selects the move with the best estimated minimax value, stopping at a cutoff depth. It uses recursive depth-first search and an evaluation function to score positions at the cutoff.

The recurrence is:

$$\text{H-MINIMAX}(s,d) = \begin{cases} \text{EVAL}(s), & \text{IS-CUTOFF}(s,d), \\ \max\limits_{a \in A(s)} \text{H-MINIMAX}(T(s,a), d+1), & \text{TO-MOVE}(s) = \text{MAX}, \\ \min\limits_{a \in A(s)} \text{H-MINIMAX}(T(s,a), d+1), & \text{TO-MOVE}(s) = \text{MIN}. \end{cases}$$

The cutoff test includes terminal states, where:

$$\boxed{\text{EVAL}(s) = \text{UTILITY}(s)}$$

> ***Evaluation Functions***

A simple chess heuristic is the material balance:

$$\text{EVAL}(s) = M_{\text{MAX}}(s) - M_{\text{MIN}}(s),$$

where each player's material score is:

$$M_p(s) = \#\text{pawns} + 3\#\text{knights} + 3\#\text{bishops} + 5\#\text{rooks} + 9\#\text{queens}.$$

Piece values also depend on position and game phase. The evaluation is an estimate, so a depth cutoff does not guarantee the optimal move.

> ***Horizon Effect***

The ***HORIZON EFFECT*** occurs when a significant outcome lies just beyond the cutoff. The agent may prefer moves that delay an unavoidable loss beyond its search depth, even if those moves make the final outcome worse.

Possible remedies are:

- **Quiescence search:** extend search until the position has no imminent changes that would strongly affect evaluation.
- **Singular extensions:** extend search for moves that are clearly better than the alternatives.

> ***Further Improvements***

- **Forward pruning:** consider only promising moves, or search less deeply through weaker moves. This can discard a good move.
- **Lookup tables:** store known moves or values for recurring positions, openings and endings.

4.1.4 Monte Carlo Tree Search (MCTS)

The algorithm estimates move quality through repeated simulations, concentrating exploration on promising alternatives. It stores a search tree with visit counts and simulation results to guide later simulations.

The approach presented in the slides is:

1. Simulate games using a random or learned move policy.
2. Estimate the win rate of candidate moves.
3. Use the simulation results to select a move.

The policy may be learned from game data or self-play. Search can allocate more simulations to selected positions.

> ***Evaluation***

Criterion	Value
Advantages	useful when the branching factor is very large

Criterion	Value
Disadvantages	estimates depend on simulation quality and available budget

4.2 Stochastic Games

A stochastic game includes random events that affect available moves or their outcomes. Randomness is represented by ***CHANCE*** nodes, which choose outcome r with probability $P(r \mid s)$.

Examples include backgammon and Yahtzee. Random outcomes may be fully observable: stochasticity does not imply hidden information.

4.2.1 Expectiminimax

The algorithm selects the move with the best expected outcome, considering both the opponent's choices and random events. It uses recursive search with maximum values at MAX-nodes, minimum values at MIN-nodes and weighted averages at CHANCE-nodes.

The terminal, MAX and MIN cases are the same as minimax. At a CHANCE-node:

$$\boxed{\mathrm{EXMINIMAX}(s) = \sum_{r \in A(s)} P(r \mid s)\,\mathrm{EXMINIMAX}(T(s,r))}$$

where:

$$\sum_{r \in A(s)} P(r \mid s) = 1.$$

> ***Evaluation Functions***

Expected values depend on the numerical utility values, not only on which outcomes rank higher. Changing the scale nonlinearly can therefore change the selected move.

Bounds on minimum and maximum utility can help prune CHANCE-nodes by bounding the contribution of unexplored outcomes.

> ***Evaluation***

Criterion	Value
Advantages	accounts for random outcomes and their probabilities
Disadvantages	chance outcomes greatly increase the search tree

For example, two dice generate 36 ordered outcomes per roll. Shallower search or ignoring unlikely outcomes can reduce cost, introducing approximation.

4.3 Partially Observable Games

A partially observable game hides some information about the current state. Card games are a typical example.

4.3.1 Averaging over Clairvoyance

The algorithm evaluates moves by solving possible states as if each were fully observable. It averages the resulting values to estimate move quality under uncertainty.

This approximation can suggest a plan that requires information the player does not actually have.

> ***Example***

- **Road A:** guarantees one gold coin.
- **Road B:** leads to a fork with two gold coins on one side and death on the other.
- **Known location:** choose Road B and then the safe side.
- **Unknown location:** the player cannot choose the safe side with certainty.

Averaging solutions that assume the safe side is known overvalues Road B.

> ***Evaluation***

Criterion	Value
Advantages	relatively inexpensive approximation

Criterion	Value
Disadvantages	ignores the player's information state; does not properly model bluffing

Belief states can represent the player's actual information, but increase time and memory requirements.

4.4 Search Strategies: Summary

Problem	Main Approach	Key Idea
Find an action sequence	state-space search	explore paths from the initial state
Optimize a configuration	local search	improve the objective value
Handle multiple action outcomes	AND-OR search	solve every possible outcome
Handle incomplete observation	belief-state search	search over sets of possible states
Explore an unknown environment	online search	interleave learning and acting
Play against an opponent	minimax	maximize the guaranteed outcome
Reduce game-tree exploration	alpha-beta pruning	discard branches that cannot affect the choice
Play with limited search depth	H-minimax	evaluate positions at a cutoff
Handle random events	expectiminimax	average chance outcomes by probability
Approximate hidden information	averaging over clairvoyance	average fully observable solutions

5 Constraint Satisfaction

A constraint satisfaction problem asks for values of variables that satisfy a set of constraints. It uses a factored representation: states are assignments of values to variables.

5.1 Problem Formulation

A constraint satisfaction problem is defined as:

$$\boxed{\text{CSP} = (X, D, C)}$$

where:

- $X = \{X_1, \dots, X_n\}$ is the set of variables.
- $D = \{D_1, \dots, D_n\}$ gives the domain D_i of each variable X_i.
- C is the set of constraints.

The goal is to find a satisfying assignment. Optimization problems additionally maximize an objective or minimize a cost; here we focus on satisfaction.

5.1.1 Constraints

A constraint is a pair:

$$\boxed{C_j = \langle \text{scope}, \text{rel} \rangle}$$

where:

- **Scope:** the variables involved in the constraint.
- **Relation:** the allowed combinations of their values.

For example, if $D_1 = D_2 = \{1, 2, 3\}$, the constraint $X_1 < X_2$ allows:

$$\text{rel} = \{(1, 2), (1, 3), (2, 3)\}.$$

5.1.2 Assignments and Solutions

A ***PARTIAL ASSIGNMENT*** assigns values to some variables, with each value belonging to its variable's domain.

An assignment is:

- **Consistent:** it violates no constraint whose scope is fully assigned.
- **Complete:** it assigns every variable.
- **A solution:** it is complete and satisfies every constraint.

$$\boxed{\text{Solution} = \text{Complete assignment} + \text{Consistency}}$$

A consistent partial assignment does not necessarily extend to a solution.

5.1.3 Examples

> ***Map Coloring***

Color Australia's regions so that neighbouring regions have different colors:

$$X = \{WA, NT, Q, NSW, V, SA, T\}, \qquad D_i = \{\text{red}, \text{green}, \text{blue}\}.$$

The constraints are:

$$\begin{aligned} &SA \neq WA, \quad SA \neq NT, \quad SA \neq Q, \\ &SA \neq NSW, \quad SA \neq V, \quad WA \neq NT, \\ &NT \neq Q, \quad Q \neq NSW, \quad NSW \neq V. \end{aligned}$$

For a binary CSP, the ***CONSTRAINT GRAPH*** has variables as nodes and binary constraints as edges. Tasmania T has no neighbours and can be colored independently.

> ***Job Scheduling***

Variables represent task start times. Constraints enforce precedence and completion deadlines.

If installing the front axle takes 10 minutes, the right front wheel can only be installed afterwards:

$$\text{AxleF} + 10 \leq \text{WheelRF}.$$

> ***Cryptarithmetic***

In the puzzle $TWO + TWO = FOUR$, letters represent digits and C_1, C_2, C_3 represent carries:

$$\begin{aligned} 2O &= R + 10C_1, \\ 2W + C_1 &= U + 10C_2, \\ 2T + C_2 &= O + 10C_3, \\ C_3 &= F. \end{aligned}$$

Letter domains are $\{0, \dots, 9\}$, carry domains are $\{0, 1\}$, and leading letters T, F cannot be zero. An $\mathrm{AllDiff}(T, W, O, F, U, R)$ constraint requires different letters to have different digits.

In the slide diagram, circles represent variables and squares represent constraints. Each square connects all variables in its scope, including constraints involving more than two variables.

5.2 Types of CSP

5.2.1 Domains

Type	Example
Finite	$D_i = \{\text{red}, \text{green}, \text{blue}\}$
Infinite discrete	$D_i = \mathbb{N}$ or a set of strings
Continuous	$D_i = \mathbb{R}$

Finite domains do not guarantee an easy problem: propositional satisfiability can be hard. Continuous domains do not guarantee a hard problem: linear programming admits polynomial-time algorithms.

5.2.2 Constraint Arity

Type	Variables Involved	Example
Unary	one	$SA \neq$ green
Binary	two	$SA \neq NSW$

Type	Variables Involved	Example
Ternary	three	$X + Y = Z$
Global	a collection of variables	$\text{AllDiff}(X_1, \dots, X_n)$

For finite-domain CSPs, higher-arity constraints can be converted to unary and binary constraints using auxiliary variables. The resulting representation may be much larger.

5.3 Local Search

Local search starts from a complete assignment that may violate constraints. It changes variable values to reduce the number of conflicts.

5.3.1 Min-Conflicts

The algorithm selects a random conflicted variable and assigns it a value that minimizes its conflicts. It stores a complete assignment and changes one variable at each step.

The value selection rule is:

$$\boxed{v \in \arg\min_{u \in D_i} \text{Conflicts}(X_i, u, s)}$$

where $\text{Conflicts}(X_i, u, s)$ counts violated constraints involving X_i if it takes value u in assignment s.

> ***Pseudocode***

The Python pseudocode keeps the abstract operations used in the slides. `choice` randomly selects an element; `conflicted` returns variables involved in violated constraints.

```
def min_conflicts(csp, max_steps):
    current = csp.random_assignment()
    for _ in range(max_steps):
        if csp.is_solution(current):
            return current
        var = choice(csp.conflicted(current))
        current[var] = min(csp.domains[var], key=lambda value:
                           csp.conflicts(var, value, current))
    return None
```

Evaluation

Criterion	Value
Complete	No, with a fixed step limit
Advantages	often effective on large scheduling problems
Disadvantages	can wander across plateaus without solving the CSP

Failure after the step limit does not prove that the CSP is unsatisfiable.

Plateaus

A plateau offers no single-variable change that reduces the number of conflicts. Possible remedies are:

- **Tabu search:** forbid returning to recently visited assignments.
- **Constraint weighting:** increase the weight of repeatedly violated constraints, making them more important in the objective.

5.4 Constraint Propagation

Constraint propagation removes values that cannot satisfy constraints. Reducing one domain may force reductions in other domains.

Propagation can run before search and after each assignment. Stronger consistency checks can remove more possibilities, but cost more to enforce.

5.4.1 Node Consistency

A variable is ***NODE-CONSISTENT*** if every value in its domain satisfies its unary constraints.

For example:

$$D_{SA} = \{\text{red}, \text{green}, \text{blue}\}, \quad SA \neq \text{red} \quad \Rightarrow \quad D_{SA} = \{\text{green}, \text{blue}\}.$$

5.4.2 Arc Consistency

A directed arc $X_i \to X_j$ is ***ARC-CONSISTENT*** if every value of X_i has a compatible value in D_j:

$$\boxed{\forall x \in D_i \; \exists y \in D_j : \quad C_{ij}(x, y)}$$

where C_{ij} represents the binary constraint between the variables. A compatible value y is a ***SUPPORT*** for x.

For example:

$$D_{SA} = \{\text{red}, \text{green}, \text{blue}\}, \quad D_{WA} = \{\text{red}\}, \quad SA \neq WA.$$

The value red has no support for SA, so it must be removed from D_{SA}.

Arc consistency is directional: checking $X_i \to X_j$ may remove values from D_i, not D_j. A binary CSP is arc-consistent when all directed arcs are consistent.

5.4.3 AC-3

The algorithm removes unsupported values, rechecking neighbouring arcs whenever a domain shrinks. It uses a queue of directed arcs to propagate domain reductions.

> ***Pseudocode***

Domains are mutable sets. `allowed(x, a, y, b)` checks the binary constraints between the two variables; `arcs()` returns both directions of every edge.

```
def ac3(csp, arcs=None):
    queue = deque(csp.arcs() if arcs is None else arcs)
    while queue:
        x, y = queue.popleft()
        if revise(csp, x, y):
            if not csp.domains[x]:
                return False
            queue.extend((z, x) for z in csp.neighbours[x] if z != y)
    return True

def revise(csp, x, y):
    unsupported = {a for a in csp.domains[x]
                   if not any(csp.allowed(x, a, y, b)
                              for b in csp.domains[y])}
    csp.domains[x] -= unsupported
    return bool(unsupported)
```

> ***Evaluation***

Criterion	Value
Time	$O(cd^3)$
Advantages	reduces domains before or during search
Disadvantages	arc consistency does not guarantee a solution

Here c is the number of binary constraints and d is the maximum domain size. Each arc is reconsidered $O(d)$ times, and each revision checks up to d^2 value pairs.

If D_i shrinks, values in neighbouring domains may lose their support. This is why AC-3 rechecks incoming arcs $X_k \to X_i$.

Assumptions:

- Finite, initially nonempty domains and constant-time binary constraint checks.
- `False` detects inconsistency; `True` means no inconsistency was found by arc consistency.

5.4.4 Stronger Consistency

- **Path consistency:** every consistent assignment to two variables extends to any specified third variable. It can remove incompatible value pairs.
- **k-consistency:** every consistent assignment to $k-1$ variables extends to any specified additional variable.
- **Global constraint propagation:** reason about a whole collection of variables together.

For example, an $\mathrm{AllDiff}$ constraint is impossible if the union of its domains contains fewer values than variables.

Two-coloring a triangle is arc-consistent, but impossible: after assigning two adjacent variables different colors, neither color remains for the third. Path consistency detects this conflict.

> ***After Propagation***

- **Empty domain:** no solution remains for the current branch.
- **All singleton domains:** check the resulting complete assignment.
- **Multiple remaining values:** search is still needed in general.

5.5 Backtracking Search

Backtracking builds a consistent partial assignment one variable at a time. Variable order affects efficiency, but does not change which complete assignments satisfy the constraints.

5.5.1 Backtracking Algorithm

The algorithm assigns a consistent value to an unassigned variable, undoing the choice when the branch fails. It uses recursive depth-first search and inference to reduce the remaining domains.

> ***Pseudocode***

`consistent` checks the candidate against the current assignment. `infer` reduces domains and returns `False` on failure; domain copies restore every reduction when backtracking.

```
def backtracking_search(csp):
    return backtrack(csp, {})

def backtrack(csp, assignment):
    if len(assignment) == len(csp.variables):
        return assignment.copy()
    var = csp.select_unassigned(assignment)
    for value in csp.order_values(var, assignment):
        if csp.consistent(var, value, assignment):
            saved = {x: values.copy() for x, values in csp.domains.
                items()}
            assignment[var] = value
            csp.domains[var] = {value}
            if csp.infer(var, assignment):
                result = backtrack(csp, assignment)
                if result is not None:
                    return result
            csp.domains = saved
            del assignment[var]
    return None
```

> ***Evaluation***

Criterion	Value
Complete	Yes, for finite domains and exhaustive value choices
Advantages	combines systematic search with pruning

Criterion	Value
Disadvantages	exponential worst-case search

There are up to d^n complete assignments for n variables with maximum domain size d. Heuristics and inference reduce how many are explored.

5.5.2 Variable Ordering

Heuristic	Selection Rule
Static ordering	use a fixed order
Random ordering	choose randomly; can be combined with restarts
Minimum Remaining Values (MRV)	choose the variable with the fewest legal values
Degree heuristic	choose the variable constraining the most unassigned neighbours

The degree heuristic is often used to break MRV ties.

Variable ordering follows ***FAIL-FIRST***: expose an impossible branch as early as possible.

5.5.3 Value Ordering

The ***LEAST CONSTRAINING VALUE (LCV)*** heuristic tries the value that removes the fewest choices from neighbouring unassigned variables.

Value ordering follows ***FAIL-LAST***: try a promising value first, since one successful value is enough.

MRV: hardest variable first	LCV: least restrictive value first

5.5.4 Inference

> ***Forward Checking***

After assigning $X_i = v$, remove values incompatible with v from each unassigned neighbour's domain. It is fast, but does not propagate conflicts between other unassigned variables.

> ***Maintaining Arc Consistency (MAC)***

After assigning $X_i = v$, propagate reductions through AC-3. Assuming the CSP was already arc-consistent, initialize the queue with:

$$\boxed{\{(X_j, X_i) \mid X_j \in N(X_i)\}}$$

The direction matters: revise neighbours against the newly restricted domain of X_i.

MAC can detect failures earlier than forward checking, but performs more inference.

For a triangle with two colors, assigning the first variable red leaves both neighbours with only blue. Forward checking finds no empty domain; MAC checks the constraint between the two neighbours and detects the failure.

5.5.5 Advanced Techniques

- **Backjumping:** identify the variables responsible for a conflict and return directly to a relevant earlier choice.
- **Constraint learning:** record failing combinations as new constraints, called ***NOGOODS***, to avoid repeating them in other branches.

5.6 Exploiting Problem Structure

5.6.1 Connected Components

Independent connected components can be solved separately. For example, Tasmania's color is independent of the rest of Australia.

With equal domain size d, two independent components of n variables each require exploring up to $2d^n$ assignments separately, rather than d^{2n} jointly.

5.6.2 Tree-Structured CSPs

A constraint graph is a ***TREE*** if it is connected and has no cycles: there is exactly one path between any two variables. A disconnected collection of trees is a ***FOREST***.

The algorithm removes unsupported parent values from the leaves towards the root, then assigns values from the root towards the leaves. It uses a rooted tree ordering to build a solution without backtracking.

Assumptions:

- Binary constraints and a tree-shaped constraint graph.
- Unary constraints have already been enforced.
- Variables are ordered with each parent before its children.

> ***Directional Arc Consistency***

Every remaining parent value must have support in each child's domain. Enforce this bottom-up by revising the **parent's domain** against the child's domain.

Once this holds, choose a root value and assign each child a value compatible with its parent. Different child subtrees have no constraints between them, so their choices can be completed independently.

Each of the $n-1$ edges is revised once, checking at most d^2 value pairs. This gives the $O(nd^2)$ running time.

> ***Pseudocode***

`tree_order` returns the parent-first order and a parent lookup table. `revise` is the same operation used by AC-3.

```
def tree_csp_solver(csp):
    order, parent = csp.tree_order()
    for child in reversed(order[1:]):
        x = parent[child]
        revise(csp, x, child)
        if not csp.domains[x]:
            return None
    assignment = {}
    for var in order:
        values = [v for v in csp.domains[var]
                  if csp.consistent(var, v, assignment)]
        if not values:
            return None
        assignment[var] = values[0]
    return assignment
```

> ***Evaluation***

Criterion	Value
Complete	Yes, under the tree assumptions
Time	$O(nd^2)$
Advantages	solves a tree CSP without backtracking
Disadvantages	general constraint graphs contain cycles

5.6.3 Cutset Conditioning

The algorithm tries assignments to a small set of variables whose removal leaves a forest. It solves the remaining tree components after restricting their domains to match each cutset assignment.

The main steps are:

1. Find a ***CYCLE CUTSET*** of variables.
2. Choose a consistent assignment to the cutset.
3. Propagate its effects to the remaining domains.
4. Solve the remaining tree CSPs; try another cutset assignment on failure.

A small cutset can greatly reduce search. Finding a minimum cutset is NP-hard, so practical methods use approximations.

5.6.4 Tree Decomposition

The algorithm groups variables into overlapping clusters arranged as a tree. It treats each cluster as a variable whose values are internally consistent assignments.

A valid decomposition must ensure that:

- Every original variable appears in a cluster.
- Every constraint's scope is contained in a cluster.
- Clusters containing the same variable form a connected subtree.

Neighbouring clusters must agree on shared variables. In the slide example, clusters $\{WA, NT, SA\}$ and $\{NT, SA, Q\}$ must assign the same colors to NT and SA.

Small clusters make the resulting tree CSP easier to solve. Finding an optimal decomposition is computationally hard.

> ***Evaluation***

Method	Main Idea	Main Cost
Cutset conditioning	assign selected variables to break cycles	enumerate cutset assignments
Tree decomposition	group variables to obtain a tree	store cluster assignments

Tree decomposition can improve time at the cost of memory exponential in cluster size. Cutset conditioning can enumerate assignments one at a time, using linear space for fixed domain size.